Gamma Lab

Informe técnico: análisis de la suite de pruebas
Qué prueba cada test, cómo funciona y qué está mal

Auditoría de código

14 de agosto de 2026

Resumen

Este informe documenta, archivo por archivo, la suite de pruebas automatizadas de **Gamma Lab** (`test/`): qué verifica cada test, cómo está construido paso a paso, y qué errores concretos se encontraron al ejecutarla realmente contra el código actual. La suite tiene tres carpetas: `services_test` (pruebas unitarias de servicios), `plugins_test` (pruebas de integración contra una referencia de MATLAB) y `performance_test` (benchmarks de rendimiento, agregados durante esta auditoría). Ningún archivo de `core/` o `plugins/` fue modificado para producir este informe; todas las cifras provienen de ejecuciones reales de `pytest` sobre el repositorio.

Índice

1. Resumen ejecutivo **2**

2. Arquitectura general de la suite **2**
2.1. Las tres capas . . . 2

3. `test/services_test/` – pruebas unitarias de servicios **3**
3.1. `test_dataset.py` – 7 tests . . . 3
3.2. `test_export.py` – 8 tests . . . 3
3.3. `test_fileio_abf.py` – 2 tests . . . 4
3.4. `test_fileio_edf.py` – 2 tests . . . 5
3.5. `test_measurement.py` – 28 tests . . . 6
3.6. `test_trials.py` – 13 tests . . . 7

4. `test/plugins_test/` – validación contra referencia de MATLAB **9**
4.1. Metodología común, paso a paso . . . 9
4.2. Tabla de tolerancias por plugin . . . 9
4.3. Error crítico #1: los 8 archivos no se pueden ni importar . . . 10
4.4. Qué se descubre al corregir el import (diagnóstico, no aplicado al repositorio) . . . 10
4.4.1. Causa raíz del fallo en `wavelet` y `wavelet_average` . . . 11
4.4.2. Fallo en `psd_average` . . . 12

5. Detalle por archivo de `test/plugins_test/` **13**
5.1. `test_average_plugin.py` . . . 13
5.2. `test_erp_plugin.py` . . . 13
5.3. `test_fft_plugin.py` . . . 13
5.4. `test_fft_average_plugin.py` . . . 13
5.5. `test_psd_plugin.py` . . . 13

5.6. test_psd_average_plugin.py 14
5.7. test_wavelet_plugin.py 14
5.8. test_wavelet_average_plugin.py 14

6. test/performance_test/ – benchmarks de rendimiento 15
6.1. Infraestructura compartida: bench_common.py 15
6.2. test_compute_bench.py – 7 tests 15
6.3. test_render_bench.py – 3 tests 16

7. Cobertura: qué no tiene ningún test 17

8. Tabla resumen de todos los errores encontrados 18

9. Recomendaciones priorizadas 18

10. Conclusión 19

1 Resumen ejecutivo

Al ejecutar `pytest` sobre el repositorio tal como está hoy, el resultado es:

Carpeta	Archivos	Tests	Estado
`test/services_test/`	6	60	**OK** (60/60 pasan)
`test/plugins_test/`	8	24	**ERROR** (0/24 se ejecutan – error de importación)
`test/performance_test/`	2	10	**OK** (10/10 pasan)

Total real medido: `70 passed, 8 errors` (los 8 errores son archivos completos que no logran ni importarse, no fallas de aserciones).

El hallazgo más importante: **los 8 archivos que validan los plugins de análisis contra una referencia de MATLAB no se ejecutan en absoluto**, por un import roto que sobrevivió a un refactor de módulos. Al corregir ese import en una copia aislada (fuera del repositorio, solo para diagnóstico – el repositorio real nunca fue modificado), 19 de 24 tests pasan y **5 fallan de verdad**, revelando errores numéricos reales en los plugins `wavelet`, `wavelet_average` y `psd_average`.

2 Arquitectura general de la suite

`test/conftest.py` es el único archivo compartido por toda la suite. Hace dos cosas:

1. Agrega la raíz del repositorio a `sys.path`, para que `import core...` y `import plugins...` funcionen sin instalar el proyecto como paquete.
2. Registra el marcador `perf` de `pytest` (agregado en esta auditoría junto con la carpeta `performance_test`, ver sección 6) para que no aparezca un *warning* de marcador desconocido.

No hay más fixtures compartidas: cada archivo de test define sus propios *helpers* y datos de prueba, lo cual produce bastante duplicación de código entre archivos (se detalla por sección).

2.1 Las tres capas

- `services_test/` – pruebas unitarias clásicas. Todo lo externo (PyQt5, VTK, `pyabf`, `pyedflib`) se sustituye por objetos *Dummy* escritos a mano o por `unittest.mock.Mock`. No dependen de archivos reales ni de una interfaz gráfica.
- `plugins_test/` – pruebas de integración. Cargan el archivo ABF real de `test/data/17308005.abf`, cortan los ensayos (*trials*) con la lógica real de `core/filters/trials.py`, ejecutan el método de cálculo real de cada plugin (usando una subclase "sin cabeza" que solo desactiva el renderizado VTK/Qt) y comparan el resultado contra un CSV de referencia generado en MATLAB.
- `performance_test/` – no verifica corrección, mide tiempo de ejecución del código real (cómputo y renderizado) para poder comparar "antes" y "después" de futuras optimizaciones.

3 `test/services_test/` – pruebas unitarias de servicios

Los 6 archivos de esta carpeta suman 60 tests. Todos pasan, todos corren en aproximadamente 1 segundo en total, y ninguno toca disco salvo para escribir archivos temporales de exportación (CSV/JSON) que luego se leen y verifican.

3.1 `test_dataset.py` – 7 tests

Qué prueba: la clase `DataStore` (`core/services/data_store.py`), el diccionario central donde se guardan las señales cargadas y cuál está "activa".

Cómo funciona, paso a paso:

1. Un *helper* `make_signal_dataset(...)` construye un `SignalDataset` real (no un mock) con señales aleatorias de NumPy, para no depender de ningún archivo.
2. Cada test crea un `DataStore()` nuevo (vacío) y ejercita una operación puntual.
3. No hay mocks: es todo objeto real, así que si `DataStore` cambia su comportamiento interno, el test lo detecta de verdad.

Test	Qué verifica
`test_set_get_has_remove_and_items`	`set/get/has/remove/items` básicos del diccionario.
`test_add_signal_with_explicit_key_and_get_signals_filters`	Al agregar una señal con clave explícita, `get_signals()` la devuelve y excluye entradas no-señal.
`test_add_signal_auto_keys_increment`	Sin clave explícita, las claves autogeneradas siguen el patrón `raw_signal_1, _2, _3...`
`test_set_active_signal_success_and_getters`	Marcar una señal como activa y leerla de vuelta con los distintos getters.
`test_set_active_signal_invalid_key_raises`	Marcar como activa una clave inexistente lanza `ValueError`.
`test_set_active_signal_non_signal_raises`	Marcar como activo algo que no es un `SignalDataset` lanza `ValueError`.
`test_clear_active_signal_and_is_active_signal`	`clear_active_signal()` deja el store sin señal activa.

Errores encontrados: ninguno. Es, junto con `test_trials.py`, el archivo mejor construido de toda la suite: sin mocks innecesarios, aserciones concretas, nombres de test que describen exactamente el caso cubierto.

3.2 `test_export.py` – 8 tests

Qué prueba: `ExportService` (`core/services/export_service.py`), que exporta el gráfico activo a imagen (PNG/JPEG/BMP/TIFF) o la tabla de datos a CSV/JSON/XLSX.

Cómo funciona, paso a paso:

1. Un *helper* `make_service(tmp_path, ...)` construye un `ExportService` real, pero con `parent`, `vtk_widget` y `alerts` reemplazados por `unittest.mock.Mock()`.
2. Para los tests de imagen, se usa `monkeypatch` para sustituir las clases de VTK (`vtkWindowToImageFilter`, `vtkPNGWriter`, etc.) por clases `Dummy*` que solo registran si fueron llamadas, sin generar un archivo real.

3. Para los tests de tabla, se construyen objetos `DummyTable`/`DummyChart`/`DummyPlot` que imitan la interfaz mínima de VTK que `export_table` necesita, y luego se **lee el archivo CSV/JSON real que quedó escrito en disco** y se compara su contenido.

Test	Qué verifica
`test_export_image_png_..._uses_png_writer_and_sets_last_dir`	Exportar PNG con nombre de archivo explícito dispara el alerta de éxito y actualiza el último directorio usado.
`test_export_image_dialog_cancel_returns_without_error`	Cancelar el diálogo de guardado no dispara ningún alerta.
`test_export_image_unsupported_format_calls_error`	Un formato no soportado (`gif`) dispara `alerts.error`.
`test_export_table_no_chart_triggers_error`	Sin gráfico activo, exportar tabla dispara error.
`test_export_table_chart_with_zero_plots_triggers_error`	Gráfico sin curvas dispara error.
`test_export_table_csv_writes_headers_and_rows`	El CSV escrito tiene el encabezado `P1_X,P1_Y,P2_X,P2_Y` y las filas correctas – **se lee el archivo real**.
`test_export_table_json_writes_records`	El JSON escrito tiene los registros y claves correctos – **se lee el archivo real**.
`test_export_table_uses_dialog_and_respects_cancel`	Cancelar el diálogo al exportar tabla no escribe nada ni dispara alertas.

Errores encontrados:

- **ADVERTENCIA** Los dos primeros tests de exportación de imagen **no verifican realmente que se haya usado el escritor PNG correcto**. El propio comentario del código lo admite: *"Our Dummy writers are indistinguishable from outside... we rely on the absence of errors"*. Es decir, el test pasaría igual si el código exportara accidentalmente con el escritor de JPEG en vez del de PNG.

Cómo mejorarlo: dar a cada clase `Dummy*Writer` un atributo de clase distinto (por ejemplo `FORMAT = "png"`) y, tras llamar a `export_image`, inspeccionar qué clase fue realmente instanciada (guardando la referencia en un contenedor mutable capturado por closure, o parcheando el constructor para que registre la instancia creada). Así el test detectaría si el mapeo formato→escritor se rompe.

3.3 `test_fileio_abf.py` – 2 tests

Qué prueba: `FileIOService.load_abf()`.
Cómo funciona, paso a paso:

1. Una clase `DummyABF` simula el objeto que devuelve la librería `pyabf`: 2 canales de 1000 muestras a 1000 Hz, canal 0 = seno de 5 Hz, canal 1 = coseno de 7 Hz.

2. `monkeypatch.setattr(fileio_mod, "pyabf", ...)` reemplaza el módulo `pyabf` completo dentro de `core.services.fileio_service` por un objeto que expone `ABF = DummyABF`.

3. Se llama a `FileIOService().load_abf(...)` de verdad, y se verifica la forma, la tasa de muestreo, los nombres de canal y valores puntuales conocidos (`sin(0)=0`, `cos(0)=1`).

4. Una segunda clase `DummyABF_Raises` simula un archivo corrupto (el constructor lanza `IOError`) y se verifica que el servicio propague la excepción.

Test	Qué verifica
`test_load_abf_valid`	Carga correcta: forma `(2,1000)`, `fs=1000.0`, nombres de canal, valores en `t=0` correctos, eje de tiempo del mismo largo que la señal.
`test_load_abf_corrupted`	Un ABF que lanza `IOError` al abrirse propaga `IOError` o `ValueError`, no lo esconde.

Errores encontrados: ninguno funcional. Es un archivo pequeño pero completo para lo que cubre.

Cómo mejorarlo: existe una clase `DummyABF_BadShape` (canales de largo distinto) definida en el archivo pero **sin ningún test que la use** – el caso "canales con longitudes inconsistentes" está escrito pero nunca se ejecuta. Agregar un tercer test que la use y documente si el loader debe recortar, rellenar o lanzar una excepción en ese caso.

3.4 `test_fileio_edf.py` – 2 tests

Qué prueba: `FileIOService.load_edf()`. Mismo patrón que el archivo ABF, pero simulando `pyedflib.EdfReader`.

Cómo funciona, paso a paso:

1. `DummyEDF_Uniform` simula un EDF válido de 2 canales, misma longitud (1000 muestras) y misma frecuencia de muestreo (1000 Hz) en ambos.
2. `monkeypatch` reemplaza `pyedflib.EdfReader` por esta clase.
3. Se verifica forma, formato, nombres de canal y unidades.
4. `DummyEDF_Raises` simula un archivo corrupto; se verifica que el servicio propague la excepción.

Test	Qué verifica
`test_load_edf_valid_uniform`	Carga correcta de un EDF con canales uniformes: forma, formato, nombres y unidades.
`test_load_edf_corrupted`	Un EDF que lanza `IOError` al abrirse propaga la excepción.

Errores encontrados:

- **ADVERTENCIA** El archivo define `DummyEDF_Inconsistent` – un EDF donde los dos canales tienen **distinta longitud y distinta frecuencia de muestreo** (1000 vs. 900 muestras, 1000 Hz vs. 500 Hz), con un comentario explícito: *"Depending on your implementation this should raise (preferred) or trigger a resampling path. Here we expect a failure."* Pero **ningún test la usa**. El caso más realista y más propenso a bugs (archivos EDF reales casi nunca tienen todos los canales a la misma frecuencia) es precisamente el que quedó sin cubrir, a pesar de que alguien ya había pensado en él y escrito el *fixture*.

Cómo mejorarlo: agregar `test_load_edf_inconsistent_channels_raises_or_resamples` usando `DummyEDF_Inconsistent`, y decidir (y documentar en el propio `fileio_service.py`) cuál de los dos comportamientos es el esperado – hoy no se sabe si el servicio maneja ese caso correctamente porque nunca se ejecutó contra él.

3.5 `test_measurement.py` – 28 tests

Qué prueba: `MeasurementService` (`core/services/measurement_service.py`), la máquina de estados que maneja las mediciones interactivas de pendiente/amplitud sobre los gráficos VTK (click en punto A, click en punto B, guardar medición).

Cómo funciona, paso a paso:

1. `create_basic_service()` construye un `MeasurementService` con `parent`, `vtk_widget`, `get_active_chart`, `ds_get` y `ds_set` todos `Mock()` – nunca toca VTK ni Qt real.
2. `create_service_with_data()` además precarga `svc._ref_data` con un par de listas `xs`/`ys` conocidas, para los tests de búsqueda de punto más cercano.
3. Cada grupo de tests ejercita una porción de la máquina de estados y luego **inspecciona atributos internos** (`svc._state`, `svc._current`, `svc._ref_axes`, etc.) directamente – es una prueba de caja blanca, no solo de la interfaz pública.
4. Para las funciones que calculan la medición en sí (`two_point_metrics`, `amplitude_in_window`), se usa `@patch(...)` para inyectar un resultado conocido y así aislar la lógica de guardado/estado de la lógica numérica (que vive en `core/filters/measurements.py` y **no tiene test propio**, ver sección 7).

Los 28 tests se agrupan en 8 bloques temáticos dentro del propio archivo:

Bloque	#	Qué cubre
Máquina de estados (`start`/`cancel`)	7	Iniciar medición de pendiente/amplitud, bloqueo por reentrada, rechazo si se pide `slope_all_trials` desde un plugin que no es `trials`, cancelar limpia todo el estado.
Persistencia (`_save_measurement`)	5	IDs autoincrementales (`slope-001, -002...`, incluso pasando de 999 a 1000), contexto (`view_id`, `trial_id`, `channel_name`) guardado y sobreescribible.
Pendientes por trial	3	`_compute_and_save_slopes_each_trial_by_index` recorre todas las columnas `trial_N` de la tabla VTK activa, calcula la pendiente de cada una y genera overlays solo para el trial actualmente mostrado.
Eliminación	3	Quitar la última medición, manejar lista vacía, limpiar todas las mediciones.
Helpers de búsqueda	4	`_nearest_index` (coincidencia exacta, interpolado, sin datos, con duplicados).
Contexto/limpieza	2	`cancel()` limpia todo el estado interno; `clear_all_measurements()` preserva el contexto (`view_id`, `trial_id`) aunque borre las mediciones.

Bloque	#	Qué cubre
Flujos de integración	3	Flujo completo de una medición de pendiente y una de amplitud de principio a fin; múltiples mediciones consecutivas incrementan el contador correctamente.

Errores encontrados: ninguno funcional en este archivo – es el más grande y, junto con `test_trials.py`, el más exhaustivo de la suite (28 tests para un solo servicio).

Cómo mejorarlo: al ser pruebas de caja blanca (dependen de nombres de atributos privados como `_state`, `_current`), son frágiles ante refactors internos de `MeasurementService` aunque el comportamiento externo no cambie. No es un error, pero conviene documentarlo como una decisión de diseño consciente: si se refactoriza el servicio, hay que actualizar estos 28 tests aunque la interfaz pública no cambie.

3.6 `test_trials.py` – 13 tests

Qué prueba: `core/filters/trials.py`, el módulo que detecta estímulos y corta la señal continua en ensayos (*trials*). Es el **único archivo de `services_test` que no usa ningún mock**: corre la lógica real sobre señales sintéticas (senos e impulsos) construidas a mano y sobre objetos `SignalDataset` reales.

Cómo funciona, paso a paso:

1. Dos generadores, `impulse_train(...)` (tren de impulsos en índices dados) y `sine(...)` (seno de frecuencia conocida), producen señales sintéticas donde el resultado esperado se puede calcular a mano.
2. `make_signal_dataset(...)` las empaqueta en un `SignalDataset` real.
3. Los tests atacan el módulo en tres niveles: funciones internas de bajo nivel (`_detect_onsets_abs`, `_compute_windows`, `_extract_trials_matrix`, `_assemble_trials_general`) y la función pública de extremo a extremo (`cut_trials_single_channel`).

Test	Qué verifica
`test_detect_onsets_abs_no_crossings`	Señal en cero no detecta ningún cruce de umbral.
`test_detect_onsets_abs_single_crossing`	Un único impulso se detecta en el índice correcto.
`test_detect_onsets_abs_multiple_crossings_mixed_sign`	Detecta cruces positivos y negativos por igual (usa valor absoluto).
`test_compute_windows_fixed_basic_clipping_and_ns`	Modo `fixed`: la ventana se recorta correctamente contra los bordes de la señal y `Ns` coincide con `(t1-t0)*fs`.
`test_compute_windows_until_next_onset_groups_and_last_to_end`	Modo `until_next_onset`: cada ventana termina donde empieza la siguiente, y la última llega hasta el final de la señal.
`test_extract_trials_matrix_alignment_and_padding`	El relleno (*padding*) con NaN al inicio/final de cada columna respeta el `lead_pad` calculado.
`test_assemble_trials_general_groups_three_per_trial`	Con 3 estímulos por ensayo, agrupa correctamente 6 columnas en 2 trials.

Test	Qué verifica
`test_cut_trials_no_stimuli_returns_single_column`	Sin estímulos detectados, devuelve 1 sola columna (toda la señal como un trial).
`test_cut_trials_forced_no_stimulus_even_with_triggers`	Con `stim_expected=0`, ignora los disparos presentes en el canal de estímulo aunque existan.
`test_cut_trials_single_stim_fixed_window_alignment_and_ns`	Con 2 estímulos, ventana fija: el valor de la señal en el instante del estímulo (índice cero relativo) coincide exactamente con la señal original.
`test_cut_trials_single_stim_until_next_onset_variable_lengths`	3 estímulos separados de forma desigual producen columnas de largo variable, rellenadas con NaN donde corresponde.
`test_cut_trials_multi_stim_assemble_ok`	2 trials de 3 estímulos cada uno se agrupan correctamente y `stim_per_trial`/`stim_detected` quedan bien en los metadatos.
`test_cut_trials_stim_channel_differs_from_target_channel`	El canal de detección de estímulo puede ser distinto del canal que se corta, y la alineación sigue siendo exacta.

Errores encontrados: ninguno. Es, con `test_dataset.py`, el ejemplo a seguir del resto de la suite: sin mocks, con valores esperados calculados de forma independiente (no copiados de la salida del propio código), y con nombres de test que documentan exactamente la condición de borde cubierta.

4 test/plugins_test/ – validación contra referencia de MATLAB

Los 8 archivos de esta carpeta comparten **exactamente la misma metodología** y los mismos parámetros de corte de ensayos – solo cambian el plugin bajo prueba y las tolerancias numéricas.

4.1 Metodología común, paso a paso

1. **Carga real:** `FileIOService().load_abf("test/data/17308005.abf")` – el mismo archivo ABF real en los 8 archivos.
2. **Corte de trials real:** `core.filters.trials.cut_trials_single_channel(...)` con los mismos parámetros en los 8 archivos: canal objetivo `0`, canal de estímulo `1`, umbral `0.7`, ventana `t0=-0.05 s` a `t1=4.00 s`, modo `until_next_onset`.
3. **Doble "sin cabeza":** se define una subclase del plugin real (p. ej. `AverageDouble(Average_plugin)`) que sobreescribe *únicamente* los métodos de creación de VTK/renderizado (`ensure_vtk`, `render_*`) para que no hagan nada, y expone un método `run()` que reproduce la lógica de cálculo real del botón "Calcular" sin pasar por la interfaz gráfica.
4. **Cálculo real:** se invoca el método de cálculo real del plugin (p. ej. `_compute_fft`, `compute_wavelet`) – ninguna fórmula matemática está mockeada.
5. **Referencia MATLAB:** se carga un CSV (`test/data/*_data_matlab.csv`) generado previamente en MATLAB con el mismo ABF y los mismos parámetros.
6. **Tres comparaciones** entre el resultado de la app y la referencia:
   (a) **Forma:** ambas matrices son 2D y no están vacías tras recortarlas al tamaño mínimo común.
   (b) **Correlación de Pearson** por columna (un trial/canal) o por fila (una frecuencia, en los plugins de tiempo-frecuencia), exigiendo un mínimo `MIN_CORR`.
   (c) **Tolerancia puntual:** al menos `FRAC_MIN_PASS` (95 % en los 8 archivos) de los puntos deben cumplir `|APP - MATLAB| <= FRAC_ATOL`, con un reporte detallado de las peores filas/columnas si falla.

4.2 Tabla de tolerancias por plugin

Plugin	Correlación mín.	Tolerancia puntual	Eje de comparación
`average`	0.98	0.001	columna (trial)
`erp`	0.98	0.001	columna
`fft`	0.98	0.75	columna
`fft_average`	0.98	0.15	columna
`psd`	0.88	0.0001	columna
`psd_average`	0.95	0.0001	columna
`wavelet`	0.80	0.0001	fila (frecuencia)
`wavelet_average`	0.80	0.0001	fila (frecuencia)

Estas tolerancias no siguen ningún criterio común: `fft` acepta un error puntual de `0.75` mientras `psd` exige `0.0001` – cuatro órdenes de magnitud de diferencia entre dos plugins que calculan cosas relacionadas (espectro de frecuencia). Todo indica que se ajustaron caso por caso hasta que el test pasara, no a partir de un presupuesto de error definido de antemano.

4.3 Error crítico #1: los 8 archivos no se pueden ni importar

ERROR Los 8 archivos contienen, en sus primeras líneas, estos dos imports:

```
from core.services.fileio import FileIOService
from core.services.trial_dataset import TrialDataset
```

Ninguno de esos dos módulos existe hoy en el repositorio. Los módulos reales son:

```
from core.services.fileio_service import FileIOService   # antes: core.services.fileio
from core.model.trial_dataset import TrialDataset         # antes: core.services.trial_dataset
```

Al ejecutar `pytest` sobre el repositorio real, esto es lo que ocurre (salida real, verificada):

```
ERROR test/plugins_test/test_average_plugin.py
ERROR test/plugins_test/test_erp_plugin.py
ERROR test/plugins_test/test_fft_average_plugin.py
ERROR test/plugins_test/test_fft_plugin.py
ERROR test/plugins_test/test_psd_average_plugin.py
ERROR test/plugins_test/test_psd_plugin.py
ERROR test/plugins_test/test_wavelet_average_plugin.py
ERROR test/plugins_test/test_wavelet_plugin.py
8 errors during collection
```

Por qué pasó: en algún momento se movió/renombró `core/services/fileio.py` a `core/services/fileio_service.py`, y se separó `TrialDataset` de `core/services/` a un módulo nuevo `core/model/trial_dataset.py`. Los 8 archivos de test nunca se actualizaron para reflejar ese refactor. Como no hay integración continua (*CI*) configurada en el repositorio, nadie lo notó: la validación contra MATLAB de 8 de los plugins más importantes de la aplicación lleva rota desde ese refactor sin que ninguna corrida automática lo haya señalado.

Cómo mejorarlo:

1. Corregir los dos imports en los 8 archivos (cambio mecánico, una línea por archivo).
2. Agregar un *workflow* de CI (por ejemplo GitHub Actions) que corra `pytest` en cada *push*/*pull request*, para que un error de este tipo se detecte en minutos y no de forma manual meses después.

4.4 Qué se descubre al corregir el import (diagnóstico, no aplicado al repositorio)

Para saber si, más allá del import, el resto de la lógica de estos 8 archivos funciona, se corrigieron esos dos imports **en una copia del código fuera del repositorio** (usada solo para diagnóstico y luego eliminada; el repositorio real de Gamma Lab nunca fue modificado). Resultado real de esa ejecución:

Archivo	Tests	Resultado	Detalle
`test_average_plugin.py`	3	**OK**	todos pasan
`test_erp_plugin.py`	3	**OK**	todos pasan
`test_fft_plugin.py`	3	**OK**	todos pasan
`test_fft_average_plugin.py`	3	**OK**	todos pasan
`test_psd_plugin.py`	3	**OK**	todos pasan
`test_psd_average_plugin.py`	3	**ADVERTENCIA**	1 falla (correlación columna 0 = 0.58, s
`test_wavelet_plugin.py`	3	**ERROR**	2 fallan (ver §4.4.1)
`test_wavelet_average_plugin.py`	3	**ERROR**	2 fallan (mismo patrón)
Total	**24**	**19 pasan / 5 fallan**	

4.4.1. Causa raíz del fallo en `wavelet` y `wavelet_average`

Los tests `test_correlation_by_row` y `test_pointwise_fraction_allowing_outliers` de ambos archivos fallan de forma severa: correlación mínima por fila de **-0.23** (se exige 0.80) y solo el **0.26 %** de los puntos dentro de tolerancia (se exige 95 %). Investigando el porqué:

1. Los tests usan por defecto `scale_log=True` (remuestreo del eje de frecuencia a escala logarítmica antes de comparar), igual que la interfaz cuando el usuario activa la casilla "Log".

2. Al desactivar únicamente ese paso (`_scale_log`, en `wavelet_plugin.py`, líneas 242–267) y comparar el escalograma *crudo* contra la misma referencia de MATLAB, la correlación media por fila sube de **0.22 a 0.75**. Esto aísla a `_scale_log` como el responsable principal del colapso: el remuestreo interpola linealmente sobre una rejilla de solo 998 puntos (˜0.5 Hz de paso), lo cual degrada gravemente la resolución cerca de 1 Hz.

3. Aun así, con el escalograma crudo (sin ese paso), el **60 % de las filas (596 de 998)** siguen por debajo del umbral de correlación, y la razón de magnitud APP/MATLAB varía fila a fila entre **0.54× y 11.3×** – no es un factor de escala constante corregible con un multiplicador único. Esto es compatible con dos causas adicionales ya identificadas en el código: la decimación de la señal (`sig[::factor]`, línea 202) **sin filtro anti-aliasing** previo, y una posible diferencia de convención de normalización entre el wavelet Morlet de `pywt.cwt` y el `cwt` de MATLAB.

Cómo mejorarlo:

1. Revisar y corregir `_scale_log` en `wavelet_plugin.py` – probablemente necesita una rejilla de interpolación más fina que la resolución lineal original, no la misma cantidad de puntos.

2. Agregar un filtro pasa-bajos antes de cualquier decimación en `compute_wavelet`.

3. Verificar contra la documentación de MATLAB qué convención de normalización usa su `cwt` para el wavelet de Morlet, y ajustar el cálculo de `pywt` para que coincida (factor de escala explícito si es necesario).

4. Una vez corregido, bajar `FRAC_ATOL` y subir `MIN_CORR` a valores acordes al resto de la suite (hoy estos dos plugins tienen, con diferencia, las tolerancias más laxas) para que el test siga siendo una guardia de regresión útil y no solo un test que "pasa como sea".

4.4.2. Fallo en `psd_average`

La columna 0 (el primer trial) tiene correlación 0.58 contra un umbral de 0.95. Es consistente con un hallazgo de corrección independiente detectado al leer el código: `psd_average_plugin.py`, línea 255, llama a `np.nan_to_num(Xds, copy=False, ...)` – cuando no hay decimación (`srt==1`), `Xds` es literalmente el mismo array en memoria que `X`, así que esta limpieza de NaN **muta el array de trials compartido en el sitio**, en vez de trabajar sobre una copia. Cualquier otro plugin que use el mismo `TrialDataset` después puede estar viendo datos ya alterados.

Cómo mejorarlo: cambiar la línea 255 a `Xds = np.nan_to_num(Xds, copy=True, ...)` (o copiar `X` antes de decimar), y agregar un test unitario en `services_test` que verifique explícitamente que `_compute_psd` no modifica su argumento de entrada.

5 Detalle por archivo de `test/plugins_test/`

Todos siguen la plantilla de la sección anterior; aquí se detalla qué calcula específicamente cada plugin y sus 3 tests.

5.1 `test_average_plugin.py`

Compara el promedio punto a punto entre trials (`np.mean` sobre el eje de trials) contra `average_data_matlab.csv`.

`test_shape_and_length`	Forma no vacía tras recorte a tamaño común.
`test_correlation_with_matlab`	Correlación $\geq 0{,}98$.
`test_pointwise_fraction_allowing_outliers`	$\geq 95\,\%$ de puntos con error $\leq 0{,}001$.

Estado real: **OK** los 3 pasan (una vez corregido el import de la sección 4.3).

5.2 `test_erp_plugin.py`

Compara la matriz *butterfly* (una fila por trial) contra `erp_data_matlab.csv`.

`test_shape_and_columns`	Forma no vacía tras recorte.
`test_correlation_by_column`	Correlación por trial $\geq 0{,}98$.
`test_pointwise_fraction_allowing_outliers`	$\geq 95\,\%$ de puntos con error $\leq 0{,}001$.

Estado real: **OK** los 3 pasan.

5.3 `test_fft_plugin.py`

Compara la magnitud de `np.fft.rfft` (sin ventana, sin normalizar) contra `fft_data_matlab.csv`.

`test_shape_and_columns`	Forma no vacía tras recorte.
`test_correlation_by_column`	Correlación por trial $\geq 0{,}98$.
`test_pointwise_fraction_allowing_outliers`	$\geq 95\,\%$ de puntos con error $\leq 0{,}75$ (tolerancia más laxa de

Estado real: **OK** los 3 pasan – pero con una tolerancia puntual de `0.75` que oculta casi cualquier discrepancia salvo las más groseras.

5.4 `test_fft_average_plugin.py`

Igual que el anterior, pero sobre el promedio de la magnitud FFT entre trials; referencia `fft_average_data_matlab.csv`. Estado real: **OK** los 3 pasan (tolerancia `0.15`).

5.5 `test_psd_plugin.py`

Compara la densidad espectral de potencia (`scipy.signal.welch`) contra `psd_data_matlab.csv`.

`test_shape_and_columns`	Forma no vacía tras recorte.
`test_correlation_by_column`	Correlación por trial $\geq 0{,}88$.
`test_pointwise_fraction_allowing_outliers`	$\geq 95\,\%$ de puntos con error $\leq 0{,}0001$.

Estado real: **OK** los 3 pasan.

5.6 `test_psd_average_plugin.py`

Igual que el anterior sobre el promedio entre trials; referencia `psd_average_data_matlab.csv`. Estado real: **ADVERTENCIA** `test_shape_and_columns` y `test_pointwise_fraction_allowing_outliers` pasan; `test_correlation_by_column` **falla** (columna 0, corr. 0.58 vs. umbral 0.95). Ver causa raíz en la sección anterior (mutación in-place de NaN).

5.7 `test_wavelet_plugin.py`

Compara el escalograma de un único trial (CWT con wavelet de Morlet vía `pywt.cwt`) contra `wavelet_data_matlab.csv`.

`test_shape_and_size`	Forma no vacía tras recorte.
`test_correlation_by_row`	Correlación por frecuencia $\geq 0{,}80$.
`test_pointwise_fraction_allowing_outliers`	$\geq 95\,\%$ de puntos con error $\leq 0{,}0001$.

Estado real: **ERROR** solo `test_shape_and_size` pasa; los otros dos fallan. Ver sección 4.4.1.

5.8 `test_wavelet_average_plugin.py`

Igual que el anterior, promediando el escalograma de todos los trials; referencia `wavelet_average_data_matlab.csv`. Estado real: **ERROR** mismo patrón de fallo que `wavelet`.

6 `test/performance_test/` – benchmarks de rendimiento

Carpeta agregada durante esta auditoría (no existía antes). No verifica corrección – mide tiempo de ejecución del código **real**, sin mocks, contra el ABF real, para poder comparar "antes" y "después" de futuras optimizaciones de rendimiento.

6.1 Infraestructura compartida: `bench_common.py`

No es un archivo de test en sí, sino el módulo de utilidades que usan los otros dos:

1. `time_block(fn, *args, reps=5, warmup=1, **kwargs)` – ejecuta `fn` una vez de calentamiento (se descarta) y luego `reps` veces cronometradas con `time.perf_counter()`; devuelve mínimo, mediana, media y máximo en segundos.
2. `record(nombre, stats, label=...)` – agrega el resultado a `results/perf_results.json`, bajo la etiqueta del *run* (variable de entorno `GAMMA_PERF_LABEL`, por defecto `"baseline"`). Etiquetas distintas no se pisan entre sí.
3. `compare_report.py` – script standalone que lee ese JSON y imprime una tabla antes/después con el factor de aceleración (*speedup*) por benchmark.

6.2 `test_compute_bench.py` – 7 tests

Qué mide: el tiempo de cómputo puro (sin VTK) de los 6 plugins de análisis. Hoy, 5 de esos 6 (`fft`, `fft_average`, `psd`, `psd_average`, `relative_psd`) y el `wavelet` simple corren **en el hilo principal de Qt**, sin `QThread` – así que este número es directamente "cuánto se congela la interfaz por cada clic en Calcular".

Cómo funciona, paso a paso:

1. *Fixtures* de módulo cargan el ABF real una sola vez y cortan los trials con los mismos parámetros que `plugins_test` (sección 4).
2. Cada test instancia el plugin real (sin interfaz gráfica) y llama a `time_block` sobre su método de cálculo real (p. ej. `plug._compute_fft`, `plug.compute_wavelet`).
3. Los resultados se imprimen en vivo con `pytest -s` y quedan guardados en el JSON.

Benchmark	Mediana medida
`fft._compute_fft` (60 trials)	3.7 ms
`fft_average._compute_fft_average` (60 trials)	8.3 ms
`psd._compute_psd` (Welch, 60 trials)	4.9 ms
`psd_average._compute_psd` (Welch, 60 trials)	9.6 ms
`relative_psd._compute_psd` (Welch, 60 trials)	5.7 ms
`relative_psd._compute_relative_psd` (razón)	0.07 ms
`wavelet.compute_wavelet` (1 trial, CWT)	**203 ms**
`wavelet_average.compute_wavelet` (×20 trials)	**4.17 s**

"Error" encontrado (de firma, no de lógica): al escribirlo, `psd_average._compute_psd` resultó tener **una firma distinta** a `psd`/`relative_psd` (no acepta el parámetro `detrend`) – confirmado al leer el código, no es un bug del test sino una inconsistencia real entre plugins que deberían ser gemelos. El benchmark se ajustó para llamarlo con su firma real.

Cómo mejorarlo (del código bajo prueba, no del test): unificar la firma de `_compute_psd` entre `psd`, `psd_average` y `relative_psd` – hoy son tres copias casi idénticas con pequeñas diferencias accidentales, lo cual además facilita que un fix se aplique a una copia y se olvide en las otras (exactamente lo que pasó con el bug de NaN de la sección 4.4).

6.3 `test_render_bench.py` – 3 tests

Qué mide: el bucle `for i... for j...: img.SetScalarComponentFromFloat(...)` que construyen `wavelet`, `wavelet_average` y `erp` para dibujar su mapa 2D en VTK, en vez de vectorizar con `vtk.util.numpy_support.numpy_to_vtk` (como ya hace `core/utils/adapters.py` para todos los gráficos de líneas de la app).

Cómo funciona, paso a paso:

1. Se instancia el plugin real y se le asignan los atributos mínimos que `render_scalogram`/ `_render_heatmap` necesitan (`vtk_widget`, `_context_view`) apuntando a una `vtkRenderWindow` en modo *offscreen* (`SetOffScreenRendering(1)`) – no se abre ninguna ventana, pero el código de renderizado que corre es exactamente el real.

2. Se cronometra la llamada real a `render_scalogram`/`_render_heatmap` con datos reales (un escalograma calculado con `compute_wavelet` sobre el ABF real).

Benchmark	Tamaño	Mediana
`wavelet.render_scalogram`	998×3051 (3.0M puntos)	**701 ms**
`wavelet_average.render_scalogram`	998×3051 (3.0M puntos)	**728 ms**
`erp._render_heatmap`	60×30501 → recortado internamente a 60×1907	34 ms

Por qué `erp` sale tan barato en comparación: `erp_plugin.py` tiene un límite interno (`MAX_SAMPLES = 2000`) que recorta el eje de tiempo *antes* de entrar al bucle, así que en la práctica solo itera sobre ∼114 000 puntos, no 1.8 millones. Sufre el mismo defecto de diseño que `wavelet`/`wavelet_average`, pero un límite de resolución ya existente lo protege de pagar el costo completo.

Cómo mejorarlo: reemplazar el doble bucle de Python en los tres archivos por `numpy_support.numpy_to_vtk(Z.ravel(order=Ç"))` seguido de `img.GetPointData().SetScalars(vtk_array)` – el mismo patrón que ya usa `core/utils/adapters.py`. Es una función existente y probada en el propio repositorio; el problema no es la ausencia de la herramienta correcta, sino que estos tres archivos nunca la adoptaron.

7 Cobertura: qué no tiene ningún test

No hay archivo de test para:

- `core/kernel.py`
- `core/plugins/loader.py`
- `core/plugins/manager.py`
- `core/plugins/interfaces.py`
- `core/services/settings_service.py`
- `core/utils/adapters.py`
- `core/utils/plugin_alerts.py`
- `core/utils/vtk_context_menu.py`
- `core/filters/measurements.py`
- `app/view/main_window.py`
- `plugins/preprocessing/prepare/filter/`
- `plugins/preprocessing/prepare/artifact_remove/`
- `plugins/preprocessing/trials/` (el plugin de UI, solo el módulo interno `core.filters.trials` tiene test)
- `plugins/io/open_signal/`
- `plugins/measure/amplitude/`
- `plugins/measure/slope/`
- `plugins/analysis/frequency/relative_psd/` (sección §7)
- `plugins/faq/help/`
- Todos los archivos `*_ui.py` de cada plugin

Sobre este último punto: `relative_psd` no tiene ningún archivo en `test/plugins_test/` – fue precisamente el plugin cuyo comportamiento revisamos manualmente en esta auditoría (el selector "Calculation Mode (GF)" no tiene ningún efecto posible sobre el resultado de Potencia Relativa, por construcción algebraica de la fórmula en `_compute_relative_psd`). Es un ejemplo concreto de cómo un hueco de cobertura se traduce directamente en un bug que llegó al usuario final sin que ningún test lo hubiera podido atrapar.

8 Tabla resumen de todos los errores encontrados

Severidad	Ubicación	Descripción
ERROR	`test/plugins_test/*.py` (8 archivos)	Import roto: `core.services.fileio` / `core.services.trial_dataset` no existen. 0 de 24 tests se ejecutan.
ERROR	`wavelet_plugin.py _scale_log`	Remuestreo a escala log degrada la correlación con MATLAB de 0.75 a 0.22; bug real de implementación, no solo de tolerancia.
ERROR	`wavelet_plugin.py compute_wavelet`	Decimación sin filtro anti-aliasing + posible desajuste de normalización Morlet vs. MATLAB; 60 % de filas por debajo del umbral incluso sin el bug de `_scale_log`.
ERROR	`psd_average_plugin.py:255`	`np.nan_to_num(Xds, copy=False)` muta en el sitio el array de trials compartido; probable causa del fallo de correlación en columna 0.
ADVERTENCIA	`test_export.py`	Los tests de exportación PNG no verifican qué escritor VTK se usó realmente, solo que no hubo excepción.
ADVERTENCIA	`test_fileio_edf.py`	`DummyEDF_Inconsistent` (canales de distinta frecuencia) está escrita pero ningún test la usa.
ADVERTENCIA	`test_fileio_abf.py`	`DummyABF_BadShape` está escrita pero ningún test la usa.
ADVERTENCIA	Tolerancias de `plugins_test`	Sin criterio común (0.0001 a 0.75); parecen ajustadas para pasar, no derivadas de un presupuesto de error.
ADVERTENCIA	`psd_average._compute_psd`	Firma distinta a `psd`/`relative_psd` (sin parámetro `detrend`) sin razón aparente.
ADVERTENCIA	Toda la suite	Sin integración continua (CI): el import roto de 8 archivos completos no se detectó automáticamente.
ADVERTENCIA	§7	12 archivos de `core/` y 8 plugins sin ningún test, incluyendo `relative_psd`, donde ya se confirmó un bug real.

9 Recomendaciones priorizadas

1. **Corregir los dos imports** en los 8 archivos de `test/plugins_test/` – cambio de una línea por archivo, restaura de inmediato el 100 % de la validación contra MATLAB.
2. **Agregar un *workflow* de CI** que corra `pytest` en cada cambio, para que este tipo de rotura se detecte en minutos, no en la siguiente auditoría manual.
3. **Investigar y corregir `_scale_log` y la decimación sin filtro** en `wavelet_plugin.py`/`wavelet_average_plugin.py` – es el fallo más severo encontrado (correlación negativa contra la referencia).
4. **Corregir la mutación in-place de NaN** en `psd_average_plugin.py:255` – copiar el array antes de limpiar.
5. **Vectorizar el renderizado** de `wavelet`, `wavelet_average` y `erp` con `numpy_to_vtk`, reutilizando el patrón que ya existe en `core/utils/adapters.py` (fix de rendimiento medido en §6: hasta 700 ms por cálculo).

6. **Cubrir con tests** el kernel, el sistema de plugins, `filter`, `artifact_remove`, `trials_plugin`, `open_signal`, `amplitude`/`slope` y, en particular, `relative_psd` – donde ya se confirmó un bug de comportamiento sin ningún test que lo hubiera atrapado.

7. **Revisar las tolerancias** de `fft`/`fft_average`/`wavelet` una vez corregidos los bugs de fondo, para que dejen de estar ajustadas "a que pase" y vuelvan a ser una guardia de regresión real.

8. **Completar los casos de borde ya escritos pero sin usar:** `DummyEDF_Inconsistent` y `DummyABF_BadShape` solo necesitan un test que los invoque.

10 Conclusión

La suite de pruebas de Gamma Lab tiene, en las partes que sí se ejecutan (`services_test` y la nueva `performance_test`), un diseño sólido: datos reales o sintéticos con resultado esperado calculado de forma independiente, mocking quirúrgico solo donde hace falta (VTK, Qt, `pyabf`, `pyedflib`), y aserciones concretas. El problema no es de diseño sino de **mantenimiento**: la capa que más importa para la credibilidad científica de la aplicación –la validación de cada plugin de análisis contra una referencia de MATLAB– lleva rota por un import desde un refactor que nadie sincronizó con los tests, y nadie lo detectó porque no hay integración continua. Al corregir ese único punto de falla, la suite deja de "pasar en silencio" y empieza a hacer su trabajo: señala tres plugins (`wavelet`, `wavelet_average`, `psd_average`) con errores numéricos reales que hoy llegan al usuario sin ninguna alerta.